

Výpočetní logika

Karel Velička

February 7, 2026

Tento dokument shrnuje klíčové koncepty pro zkoušku ze SAT Solvingu. Vychází z přednášek o SAT Encodingu, CDCL, UIP a poskytnutých zkouškových řešení.

Obsah

1	SAT and Modeling with SAT	2
1.1	Základní hradla a Tseitinova transformace	2
1.2	Kódování čísel a aritmetika	2
1.3	Kardinalitní omezení (Cardinality Constraints)	2
1.4	CDCL Algoritmus a Unit Propagation	3
1.4.1	Unit Propagation (Jednotková propagace)	3
1.4.2	Implikační graf	4
1.4.3	1-UIP (First Unique Implication Point)	4
1.4.4	Algoritmus pro naučení klauzule (Rezoluce)	4
2	First-Order Logic and SMT	5
2.1	First-Order Logic (FOL) – Teorie a Modeling	5
2.1.1	Kardinalita Modelů	5
2.1.2	Důkaz sporem (Refutation)	5
2.2	SMT a Teorie	6
2.3	Teorie EUF (Equality with Uninterpreted Functions)	6
2.3.1	Congruence Closure (Algoritmus)	6
2.4	Teorie IDL (Integer Difference Logic)	7
2.4.1	Konstrukce grafu a algoritmus	7
2.5	Lazy SMT Workflow (DPLL(T))	8
3	Quantifier Elimination	9
3.1	Linear Real Arithmetic (LRA)	9
3.1.1	Fourier-Motzkinova Eliminace	9
3.2	Linear Integer Arithmetic (LIA) - Cooperova Metoda	9
3.2.1	Krok 1: Normalizace koeficientů (LCM trick)	9
3.2.2	Krok 2: Eliminace (Cooper's Expansion)	10
3.3	Řešené příklady ze zkoušek (III-QE.pdf)	10

1 SAT and Modeling with SAT

Cílem je převést problém reálného světa (nebo logický obvod) do **CNF** (Conjunctive Normal Form). CNF je konjunkce klauzulí, kde každá klauzule je disjunkcí literálů. Důležitý je přepis implikace na $\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$.

1.1 Základní hradla a Tseitinova transformace

Tseitinova transformace se využívá v momentě, kdy máme *obecnou výrokovou formuli* nebo *logický obvod s hradly* a potřebujeme ji převést do CNF, kterou vyžadují SAT solvery. Použijeme ji, když "naivní" převod pomocí distributivních zákonů a základních pravidel vede k exponenciálnímu nárůstu počtu klauzulí.

Místo abychom složitou formuli roznásobovali, zavedeme pro každý podvýraz *novou pomocnou proměnnou*. Pokud máme rovnici $z = OP(x, y)$, musíme zakódovat ekvivalenci $z \leftrightarrow OP(x, y)$.

- **NOT** ($z = \neg a$):

$$(a \vee z) \wedge (\neg a \vee \neg z)$$

- **OR** ($z = a \vee b$): Implikace $a \vee b \rightarrow z$ a $z \rightarrow a \vee b$.

$$(\neg a \vee z) \wedge (\neg b \vee z) \wedge (a \vee b \vee \neg z)$$

- **AND** ($z = a \wedge b$): Implikace $z \rightarrow a \wedge b$ a $z \wedge b \rightarrow z$.

$$(\neg z \vee a) \wedge (\neg z \vee b) \wedge (\neg a \vee \neg b \vee z)$$

- **XOR** ($Z = a \oplus b$): Ekvivalentní $z \leftrightarrow \neg(a \leftrightarrow b)$.

$$(\neg a \vee \neg b \vee \neg z) \wedge (a \vee b \vee \neg z) \wedge (a \vee \neg b \vee z) \wedge (\neg a \vee b \vee z)$$

- **Ekvivalence** ($a \leftrightarrow b$):

$$(\neg a \vee b) \wedge (\neg b \vee a)$$

1.2 Kódování čísel a aritmetika

Čísla jsou obvykle kódována binárně pomocí n proměnných (bitů).

- **Rovnost čísel** ($A = B$): Platí, pokud se rovnají všechny bity na odpovídajících pozicích.

$$\bigwedge_{i=0}^n (a_i \leftrightarrow b_i)$$

- **Sčítání**: Pro sčítání bitů a_i, b_i a carry c_{in} získáme sumu s_i a carry-out c_{out} .

$$s_i = a_i \oplus b_i \oplus c_{in}$$

$$c_{out} = (a_i \wedge b_i) \vee (c_{in} \wedge (a_i \oplus b_i))$$

1.3 Kardinalitní omezení (Cardinality Constraints)

Tato omezení slouží k zakódování podmínek na počet pravdivých proměnných v množině $x_1, \dots, x_n$. Tedy například pro omezení typu "nejvýše k z n proměnných je true" ($\sum x_i \leq k$).

At Least One (Alespoň jedna). $\sum x_i \geq 1$

Toto je nejjednodušší omezení. Znamená to, že minimálně jedna proměnná musí být **True**. Kódování je jedna dlouhá klauzule obsahující všechny literály.

$$(x_1 \vee x_2 \vee \dots \vee x_n) = \bigvee_{i=1}^n x_i$$

At Most One (Nejvýše jedna). $\sum x_i \leq 1$

Buď není pravda žádná, nebo maximálně jedna. Existuje více způsobů kódování, ukážeme si ten nejjednodušší (složitost je $\mathcal{O}(n^2)$), tzv. *Pairwise encoding*.

To zakazuje, aby byly pravdivé jakékoli dvě proměnné současně. Tedy pro každou dvojici (x_i, x_j) vytvoříme klauzuli $(\neg x_i \vee \neg x_j)$:

$$\bigwedge_{i < j} (\neg x_i \vee \neg x_j)$$

Exactly One (Právě jedna). $\sum x_i = 1$

Toto omezení se skládá ze dvou částí: "Alespoň jedna" & "Nejvýše jedna".

$$(\text{AtLeastOne}) \wedge (\text{AtMostOne}) \equiv \bigvee_{i=1}^n x_i \wedge \bigwedge_{i < j} (\neg x_i \vee \neg x_j)$$

1.4 CDCL Algoritmus a Unit Propagation

Zkouškové úlohy často vyžadují simulaci CDCL (Conflict-Driven Clause Learning). Klíčové je pochopení **Implikačního grafu** a **UIP**.

1.4.1 Unit Propagation (Jednotková propagace)

Jednotková propagace se aplikuje na klauzule (disjunkce literálů) v CNF. Mějme například:

$$(p) \wedge (\neg p \vee q) \wedge (\neg r \vee \neg s) \wedge (\neg q \vee r)$$

Funguje takto:

- Jednotková klauzule je klauzule obsahující právě jeden literál (na příkladu (p)).
- Pokud takovou klauzuli najdeme, musí být tento literál nastaven na **True**, aby celá formule byla splnitelná.
- Toto přiřazení se poté propaguje do všech ostatních klauzulí.

V našem příkladu tedy propagujeme $p = \text{True}$ do všech klauzulí.

$(0 \vee q) \wedge (\neg r \vee \neg s) \wedge (\neg q \vee r)$	// $\neg p = \text{False}$, redukce na $q = \text{True}$
$(\neg r \vee \neg s) \wedge (0 \vee r)$	// $\neg q = \text{False}$, redukce na $r = \text{True}$
$(0 \vee \neg s)$	// $\neg r = \text{False}$, redukce na $s = \text{False}$

Máme tedy $\{p = \text{True}, q = \text{True}, r = \text{True}, s = \text{False}\}$.

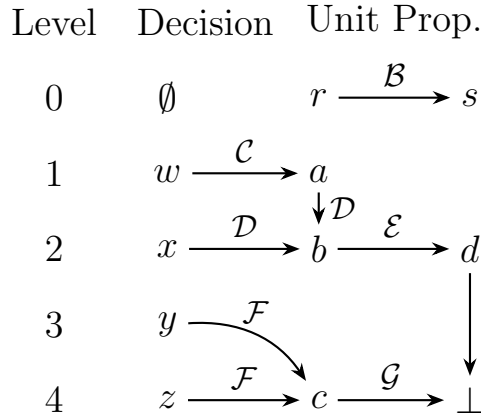
1.4.2 Implikační graf

Grafický záznam rozhodnutí a propagací.

- **Uzly:** Literály přiřazené na určitém Decision Levelu (DL).
- **Hrany:** Vychází z důvodů (antecedentů) a směřují k implikovanému literálu.
- **Konflikt ($\perp$):** Nastane, když je proměnná nucena být True i False zároveň.

Například, formule $\varphi = \underbrace{(r)}_A \wedge \underbrace{(\bar{r} \vee s)}_B \wedge \underbrace{(\bar{w} \vee a)}_C \wedge \underbrace{(\bar{x} \vee \bar{a} \vee b)}_D \wedge \underbrace{(\bar{b} \vee d)}_E \wedge \underbrace{(\bar{y} \vee \bar{z} \vee c)}_F \wedge \underbrace{(\bar{c} \vee \bar{d})}_G$.

Implikační graf pak vypadá:



1.4.3 1-UIP (First Unique Implication Point)

UIP je dominátor v implikačním grafu na aktuálním decision levelu, přes který jdou všechny cesty ke konfliktu. Důležitý pojem je “**1-UIP**”, tedy UIP je nejbližší ke konfliktu – je to poslední uzel v grafu na aktuální úrovni rozhodování předtím, než se cesty rozbíhají do konfliktu.

V našem příkladě implikačního grafu je 1-UIP uzel c . Vyskytuje se na aktuální úrovni rozhodování (Level 4), přes který procházejí všechny cesty od rozhodovací proměnné (z) ke konfliktu, a který je ke konfliktu nejbližší.

1.4.4 Algoritmus pro naučení klauzule (Rezoluce)

Postup “how is 1-UIP clause learned”:

1. Začneme s *konfliktní klauzulí* a aplikujeme pravidlo *rezoluce* zpětně proti směru propagace.
2. Rezolvujeme literál, který byl naposledy přiřazen na *aktuálním decision levelu*.
3. Opakujeme, dokud v klauzuli nezůstane *právě jeden* literál z aktuálního decision levelu. Tato klauzule je *Learned Clause* (1-UIP klauzule).

Pravidlo rezoluce:

$$\frac{(A \vee x) \quad (B \vee \neg x)}{(A \vee B)}$$

Na našem příkladu implikačního grafu je konfliktní klauzule $\mathcal{G}$: $(\neg c \vee \neg d)$. Je zde konflikt Levelu 4 a Levelu 2. Cílem je provádět rezoluci směrem zpět (od konfliktu), dokud v klauzuli nezůstane **právě jeden** literál z aktuálního decision levelu (DL 4). Vidíme, že jediným literálem z DL 4 je $\{\neg c\}$, tedy uzel c je 1-UIP. Není potřeba provádět žádné další rezoluční kroky s proměnnou c (klauzulí $\mathcal{F}$). Naučená klauzule (Learned Clause) je shodná s konfliktní klauzulí $(\neg c \vee \neg d)$.

2 First-Order Logic and SMT

Tento dokument pokrývá druhou část zkoušky: **First-Order Logic (FOL)** a **Satisfiability Modulo Theories (SMT)**. Vychází z přednášek 5 až 8 a zaměřuje se na typové úlohy:

- Konstrukce formulí s modely specifické kardinality.
- Congruence Closure algoritmus (EUF teorie).
- Integer Difference Logic (IDL) a hledání záporných cyklů.
- Lazy SMT přístup (interakce SAT a Theory solveru).

2.1 First-Order Logic (FOL) – Teorie a Modeling

Pro zkoušku je klíčové umět "programovat" ve FOL, tedy psát formule, které vynucují určitou strukturu modelu (např. kardinalitu).

2.1.1 Kardinalita Modelů

Častou otázkou je: "*Napište formuli, která má modely pouze kardinality X .*"

- *Omezení velikosti shora (At Most N)*: Pokud chceme omezit velikost univerza na maximálně N prvků, použijeme disjunkci rovností. Říkáme: "Každý prvek x musí být roven jedné z N konstant."
- *Omezení velikosti zdola (At Least N)*: Abychom vynutili minimálně N prvků, musíme říct, že existuje N vzájemně různých prvků.

$$\exists x_1 \dots \exists x_N \left(\bigwedge_{1 \leq i < j \leq N} x_i \not\approx x_j \right)$$

- *Specifické (sudé/liché) kardinality*: Pokud máme funkci f , můžeme vynutit strukturu grafu.

Příklad (Modely velikosti maximálně 3). *Mějme konstanty a, b, c .*

$$\forall x (x \approx a \vee x \approx b \vee x \approx c)$$

Každý prvek univerza se musí "trefit" do jedné z těchto konstant. Konstanty a, b, c mohou splývat ($a = b$), proto velikost je 1, 2 nebo 3.

Příklad (Modely pouze sudé kardinality). *Mějme formuli $\forall x (f(x) \not\approx x \wedge f(f(x)) \approx x)$.*

- $f(x) \not\approx x$: Neexistují smyčky délky 1.
- $f(f(x)) \approx x$: Aplikace f dvakrát vrací zpět (cyklus délky 2).

Celé univerzum se rozpadne na disjunktní cykly délky 2. Celková velikost musí být součtem dvojic $\rightarrow$ sudé číslo.

2.1.2 Důkaz sporem (Refutation)

Chceme-li dokázat, že $Axioms \models Conjecture$, postupujeme sporem:

1. Znegujeme domněnku: $Formula = Axioms \wedge \neg Conjecture$.
2. Převédeme do *Prenexní Normalní Formy (PNF)* (kvantifikátory ven).
3. Provedeme *Skolemizaci* (odstranění existenčních kvantifikátorů).
4. Pokud je výsledná množina klauzulí nespílitelná, původní tvrzení platí.

Skolemizace:

$$\forall x \exists y P(x, y) \xrightarrow{\text{skolem}} \forall x P(x, f(x))$$

Kde f je nový funkční symbol (Skolemova funkce). Pokud je $\exists$ na začátku ($\exists y P(y)$), nahrazuje se konstantou ($P(c)$).

2.2 SMT a Teorie

SMT řeší splnitelnost formulí v kontextu teorií (aritmetika, rovnost, pole). Existují dva přístupy:

- Eager (Dychtivý): Celý problém se přeloží do SAT (CNF) a pustí se SAT solver.
 - *Ackermannization*: Odstranění funkcí. Pro každé volání $f(a), f(b)$ přidáme lemma
$$a = b \implies f_a = f_b.$$
 - *Bit-blasting*: Pro bitvektory.
- Lazy (Líný): Používá se DPLL(T).
 - SAT solver řeší Booleovskou kostru (Abstract boolean structure).
 - Theory solver kontroluje konzistenci sady literálů, které SAT solver označil jako True.
 - Pokud Theory solver najde spor, vrátí SAT solveru *Blocking Clause* (vysvětlení konfliktu).

2.3 Teorie EUF (Equality with Uninterpreted Functions)

Základní teorie pro abstrakci programů. Syntaxe: $a \approx b, f(a) \not\approx c$. Uvedeme si algoritmus pro rozhodování – Congruence closure.

2.3.1 Congruence Closure (Algoritmus)

Mějme množinu rovností E a nerovností D .

1. Každý podterm (např. $a, f(a), f(f(a))$) dej do vlastní třídy ekvivalence $\{a\}, \{f(a)\}, \dots$
2. Pro každou rovnost $s \approx t \in E$: Slouč třídy obsahující s a t .
3. *Propagace kongruence*: Pokud pro nějaké funkce platí, že jejich argumenty jsou ve stejných třídách ($s_i \sim t_i$), pak slouč i výsledky funkcí ($f(s) \sim f(t)$).
4. Opakuj krok 3, dokud se něco mění.
5. *Kontrola sporu*: Pokud existuje $u \not\approx v \in D$ a u, v jsou ve stejné třídě, vrať **UNSAT**. Jinak SAT.

Příklad (Zkoušková úloha - EUF). *Rozhodněte o splnitelnosti:*

$$f(f(a)) \not\approx a \wedge c \approx f(a) \wedge a \approx c$$

Řešení. Pozitivní rovnosti: $c \approx f(a), a \approx c$. Nerovnost: $f(f(a)) \not\approx a$.

$\{a\}, \{c\}, \{f(a)\}, \{f(f(a))\}$	$\triangleright$ počáteční třídy
$\{a, c\}, \{f(a)\}, \{f(f(a))\}$	$\triangleright$ zpracuj $a \approx c$
$\{a, c, f(a)\}, \{f(f(a))\}$	$\triangleright$ zpracuj $c \approx f(a)$
$\{a, c, f(a), f(f(a))\}$	$\triangleright a \sim c \sim f(a), \text{ musí } f(a) \sim f(f(a))$

Máme $f(f(a)) \not\approx a$, ale v naší struktuře $f(f(a)) \sim a$. **UNSAT**.

2.4 Teorie IDL (Integer Difference Logic)

Omezená aritmetika, pouze nerovnosti tvaru $x - y \leq c$. Algoritmus: *Hledání záporného cyklu v grafu*.

2.4.1 Konstrukce grafu a algoritmus

Pro každou proměnnou vytvoř vrchol grafu. Nerovnost $x - y \leq c$ převed na hranu:

$$y \xrightarrow{c} x$$

Logicky, $x \leq y + c$, tedy cesta k x je omezena cestou k y plus c . Hrana $y \rightarrow x$ s váhou c v Bellman-Fordovi udržuje $dist[x] \leq dist[y] + c$.

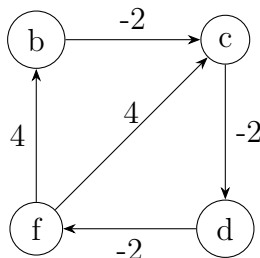
Postup:

1. Normalizuj nerovnosti na tvar $x - y \leq c$.
 - $x - y < c \implies x - y \leq c - 1$ (v celých číslech).
 - $x - y \geq c \implies y - x \leq -c$.
2. Sestroj graf.
3. Pokud graf obsahuje cyklus se záporným součtem vah, soustava je **UNSAT**.

Příklad (Zkoušková úloha - IDL). *Rozhodněte o splnitelnosti konjunkce:*

$$(c - b \leq -2) \wedge (d - c \leq -2) \wedge (f - d \leq -2) \wedge (c - f \leq 4) \wedge (b - f < 5).$$

Řešení. Nejprve převedeme poslední nerovnici do typického tvaru $b - f < 5 \rightsquigarrow b - f \leq 4$. Sestrojíme tedy dle nerovností graf:



Nyní naleumeme cyklus. Když vyzkoušíme $b \rightarrow c \rightarrow d \rightarrow f \rightarrow b$, zjistíme, že váhy jsou $(-2) + (-2) + (-2) + (4) = -2$. Tedy součet je záporný (-2) a problém je UNSAT.

2.5 Lazy SMT Workflow (DPLL(T))

Typická úloha typu "Ukažte průběh řešení pomocí Lazy SMT."

Postup:

1. *Boolean Abstraction*: Nahraď atomy ($x - y \leq 3$, $f(a) = b$) výrokovými proměnnými ($p_1, p_2 \dots$).
2. *SAT Solver*: Najdi model pro výrokovou kostru (např. $p_1 = \text{True}, p_2 = \text{False}$).
3. *Theory Check*: Vezmi literály odpovídající modelu (např. $\{x - y \leq 3, f(a) \neq b\}$) a ověř jejich splnitelnost Theory solverem (CC nebo grafem).
4. *Validace*:
 - Pokud Theory solver řekne SAT $\rightarrow$ Konec, máme řešení.
 - Pokud Theory solver řekne UNSAT $\rightarrow$ Vytvoř *blokující klauzuli* (negace současného přiřazení), přidej ji do SAT solveru a jdi na krok 2.

Příklad (Zkoušková úloha - Lazy SMT). *Mějme formuli: $(A \vee B) \wedge \neg C$, kde:*

$$A : f^2(c) \approx c, \quad B : f^3(c) \approx c, \quad C : f^6(c) \approx c.$$

1. *SAT Solver*: Zvolí model $\{A = \text{True}, B = \text{False}, C = \text{False}\}$. Tedy testujeme teorii:

$$\{f^2(c) \approx c, f^3(c) \not\approx c, f^6(c) \not\approx c\}.$$

Theory solver (Congruence Closure) zjistí, že $f^2(c) \approx c \implies f^6(c) \approx c$. To je ve sporu s $f^6(c) \not\approx c$. **Výsledek:** UNSAT. **Blocking Clause:** $\neg A \vee B \vee C$. Přidáme do SATu.

2. *SAT Solver*: Musí splnit blocking clause i původní formuli. Zvolí např. $\{A = \text{False}, B = \text{True}, C = \text{False}\}$. Tedy testujeme: $\{f^2(c) \not\approx c, f^3(c) \approx c, f^6(c) \not\approx c\}$. Theory solver zjistí, že $f^3(c) \approx c \implies f^6(c) \approx c$. Spor s $f^6(c) \not\approx c$. **Výsledek:** UNSAT. **Blocking Clause:** $A \vee \neg B \vee C$.
3. *SAT Solver*: Nemá další možnosti (kombinace A, B vedou ke sporu s C , a negace obou nesplní první klauzuli). **Závěr:** UNSAT.

3 Quantifier Elimination

Quantifier Elimination (QE) je proces, který převede formuli s kvantifikátory ($\exists x, \forall x$) na ekvivalentní formuli **bez kvantifikátorů**

- Pokud vstupní formule neobsahuje volné proměnné, výsledkem QE je $\top$ (pravda) nebo $\perp$ (nepravda).
- QE je možné pouze pro určité teorie, např. LRA (Lineární reálná aritmetika) a LIA (Lineární celočíselná aritmetika).

3.1 Linear Real Arithmetic (LRA)

V reálných číslech ($\mathbb{R}$) je situace jednodušší díky hustotě (mezi každými dvěma čísly je další číslo).

3.1.1 Fourier-Motzkinova Eliminace

Základní metoda pro eliminaci $\exists x$ v konjunkci lineárních nerovností.

Postup:

1. Vyjádři všechny nerovnosti ve tvaru $x < U_j$ (horní meze) nebo $L_i < x$ (dolní meze).
2. Pro eliminaci $\exists x(\bigwedge_i L_i < x \wedge \bigwedge_j x < U_j)$ stačí zaručit, že **všechny dolní meze jsou menší než všechny horní meze**.
3. Výsledná formule:

$$\bigwedge_{i,j} (L_i < U_j)$$

Ferrante-Rackoff (Vylepšení): Místo párování všech mezí testuje konečnou množinu bodů: $-\infty, +\infty$ a aritmetické průměry mezi mezemi $\frac{L_i + U_j}{2}$.

3.2 Linear Integer Arithmetic (LIA) - Cooperova Metoda

V celých číslech ($\mathbb{Z}$) Fourier-Motzkin nefunguje, protože mezi $3 < x$ a $x < 4$ není žádné celé číslo (ale $3 < 4$ platí)[cite: 5]. Používá se **Cooperova metoda**.

3.2.1 Krok 1: Normalizace koeficientů (LCM trick)

Abychom mohli eliminovat x , musí mít u sebe koeficient 1 (resp. -1). Pokud má rovnice tvar $ax < b$, musíme ji upravit.

Algoritmus:

1. Najdi nejmenší společný násobek (LCM) všech koeficientů u x ve všech nerovnostech.
2. Vynásob každou nerovnost tak, aby koeficient u x byl LCM .
3. Substituu $x' = LCM \cdot x$.
4. Přidej podmínku dělitelnosti: $LCM \mid x'$.

Příklad (Ze zkoušky - Normalizace). *Mějme formuli: $l < 3x \wedge 5x < u$.*
Řešení.

1. Koeficienty u x jsou 3 a 5. $LCM(3, 5) = 15$.

2. Vynásobíme první nerovnost 5 a druhou 3:

$$5l < 15x \wedge 15x < 3u$$

3. Substituce $x' = 15x$. Přidáme podmínku dělitelnosti $15 \mid x'$:

$$15 \mid x' \wedge 5l < x' \wedge x' < 3u$$

4. Nyní je koeficient u x' roven 1 a můžeme pokračovat v eliminaci.

3.2.2 Krok 2: Eliminace (Cooper's Expansion)

Mějme formuli $\exists x.\varphi(x)$, kde x má koeficient 1. Formule může obsahovat:

- Dolní meze (A): $a < x, a \leq x, x = a \dots$
- Horní meze (B): $x < b, \dots$
- Dělitelnost: $k \mid (x + c)$

Cooperova metoda říká, že $\exists x.\varphi(x)$ je ekvivalentní disjunkci dvou případů:

1. *Left Infinite Projection* ($F_{-\infty}$): Co se děje "hodně vlevo" ($v - \infty$).

- $x < b \rightarrow \top$ (je to pravda pro dost malé x)
- $a < x \rightarrow \perp$ (není pravda pro dost malé x)
- Dělitelnosti zůstávají.

2. *Boundary Check*: Řešení může existovat těsně u nějaké dolní meze $a \in A$.

Finální formule pro eliminaci: Nechť δ je LCM všech dělitelů v podmínkách dělitelnosti $k \mid \dots$

$$\exists x.F(x) \iff \bigvee_{j=1}^{\delta} F_{-\infty}(j) \vee \bigvee_{j=1}^{\delta} \bigvee_{a \in A} F(a + j)$$

Vysvětlení:

- První část ($\bigvee F_{-\infty}$) kontroluje, zda je formule splnitelná pro velmi malá čísla, přičemž musíme otestovat celou periodu δ (kvůli dělitelnosti).
- Druhá část kontroluje okolí dolních mezí. Pokud řešení existuje, musí být "kousek" (do vzdálenosti δ) od nějaké dolní meze a .

3.3 Řešené příklady ze zkoušek (III-QE.pdf)

Příklad (Konstrukce QE forem). *Zkonstruuujte formu po eliminaci kvantifikátorů (výsledek).*

(a) *Integers* $\exists x.y = 3x$, (b) *Reals*: $\exists x.l < x \wedge x < u$, (c) *Reals*: $\exists x.x^2 \leq y$. *Řešení.*

- (a) $3 \mid y$ (y je dělitelné třemi). Existuje celé číslo x takové, že y je jeho trojnásobek, právě tehdy, když je y dělitelné 3.
- (b) $l < u$. V reálných číslech existuje prvek ostře mezi l a u právě tehdy, když je interval neprázdný ($l < u$). Pozor: V celých číslech by to bylo $l + 1 < u$.
- (c) $0 \leq y$. Čtverec reálného čísla je vždy nezáporný. Pokud y je alespoň 0, můžeme najít $x = \sqrt{y}$.

Příklad (Cooperova eliminace - Expansion). Předpokládejme NNF formuli F v LIA, která obsahuje literály $a < x, x < b, 2 \mid x, 3 \mid x$. Jak Cooperova metoda eliminuje x ?
Řešení.

1. Určíme periodu δ : Máme dělitelnosti 2 a 3. $LCM(2, 3) = 6$. Tedy $\delta = 6$.
2. Identifikujeme dolní meze (A set): Literál $a < x$ dává dolní mez a .
3. Sestavíme disjunkci podle vzorce:

$$\bigvee_{j=1}^6 F_{-\infty}[x \mapsto j] \vee \bigvee_{j=1}^6 F[x \mapsto a + j]$$

(Poznámka: Ve výrazu $F_{-\infty}$ nahrazujeme x číslem j pouze v podmínkách dělitelnosti, nerovnosti jsou nahrazeny $\top/\perp$.)

Příklad (Cooperova metoda - LCM trick). Převeďte formuli $l < 3x \wedge 5x < u$ na ekvivalentní, kde koeficient u x je 1.
Řešení.

$$15 \mid x' \wedge 5l < x' \wedge x' < 3u$$

(Kde x' reprezentuje původní $15x$).